AGENT WORKFLOW DOCUMENTATION

Customer Churn Prediction & Agentic Retention Strategy

Telecommunications — Customer Intelligence

Milestone 1	Milestone 2
XGBoost · Scikit-learn · Streamlit	LangGraph · FAISS · Groq · LLaMA 3.3

TEAM MEMBERS

Vanshika Yadav — ML Modeling · Business Context · Input-Output Design · Structured Output

Ronit Singh — LangGraph Workflow · State Management · System Architecture

Riya Garg — EDA · Visualization · UI Development · Deployment

Sankalp — RAG (FAISS) · Prompt Engineering · Testing · Model Evaluation

April 2026 · Document Version 1.0 · Python · Streamlit · XGBoost · LangGraph · FAISS · Groq

Table of Contents

1. Introduction

The **Customer Churn Intelligence System** is a production-grade application that unifies classical machine learning with agentic AI reasoning. Built on the IBM Telco Customer Churn dataset, the system moves beyond simple binary classification to deliver **actionable, source-backed retention strategies** through an autonomous multi-node agent pipeline deployed as a Streamlit web application.

The system is structured around two tightly integrated milestones, each building on the output of the previous:

Milestone	Capability	Technology Stack
Milestone 1	Churn probability prediction + binary classification	XGBoost, Scikit-learn, Streamlit
Milestone 2	Agentic retention strategy generation	LangGraph, FAISS, HuggingFace Embeddings, Groq (LLaMA 3.3 70B)

This document provides a complete technical walkthrough of the **Milestone 2 agent workflow** — the agentic AI system that transforms a raw churn probability into a structured, LLM-generated retention plan grounded in retrieved domain knowledge. Every component decision — from LangGraph over vanilla LangChain, to FAISS over Pinecone, to a rule-based risk node over a secondary ML model — is documented with its rationale.

2. Objective of the Agent

The agent's primary objective is to **convert a numerical churn probability into a human-readable, actionable retention strategy** — autonomously, with no manual intervention. It must do so in a way that is trustworthy, explainable, and grounded in verified domain knowledge.

Core Responsibilities

- **Classify risk** — Interpret the raw churn probability (0.0–1.0) into a categorical risk level (High / Medium / Low) and identify the underlying reasons contributing to that risk.
- **Retrieve relevant strategies** — Use Retrieval-Augmented Generation (RAG) to fetch domain-specific retention strategies from a curated knowledge base, ensuring the output is grounded in real-world best practices rather than hallucinated content.
- **Generate a structured plan** — Invoke a Large Language Model (LLM) to synthesise the risk assessment and retrieved strategies into a coherent, structured JSON output containing a risk summary, specific recommendations, verified source citations, and a probabilistic disclaimer.

★ **Core Design Principle:** The agent must never fabricate strategies. Every recommendation must trace back to a retrieved source document, enforced through explicit prompt constraints that instruct the LLM to use ONLY provided strategies and never generate new ones.

3. High-Level Workflow

The system executes as a **linear, three-stage pipeline** orchestrated by LangGraph. The pipeline is **deterministic in structure** (the same three nodes always execute in the same order) but **adaptive in content** (the risk level, retrieved strategies, and LLM output vary based on each customer's profile).

Figure 1 — ML Inference Pipeline

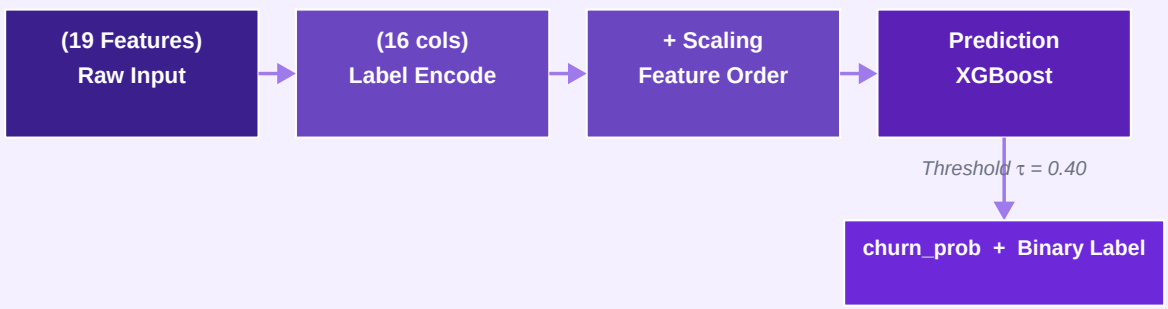

Figure 1 — ML Inference Pipeline: Raw input through encoding, scaling, and XGBoost to produce churn probability and binary label.

The ML inference pipeline (Milestone 1) produces a single floating-point churn probability. This value, combined with two customer attributes (tenure and monthly charges), is handed off to the LangGraph agentic pipeline (Milestone 2) which executes three sequential nodes: Risk Assessment → Strategy Retrieval → Retention Planning.

4. System Architecture

The system is organized into six logical layers, each with a distinct responsibility and clear interface boundaries. This layered design ensures separation of concerns, testability, and extensibility.

Figure 3 — System Architecture (Layered View)

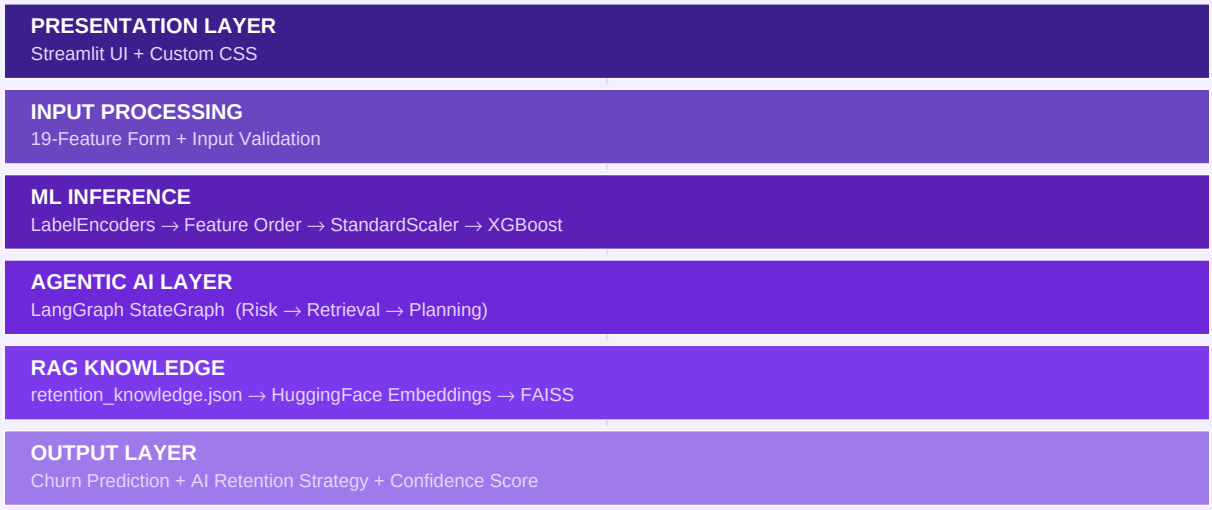

Figure 3 — Layered System Architecture from Presentation to Output.

Technology Stack

Layer	Technology	Version / Model	Purpose
Frontend	Streamlit	≥ 1.32.0	Interactive web UI
ML Model	XGBoost	≥ 1.7.0	Binary churn classification
Preprocessing	Scikit-learn	≥ 1.2.0	LabelEncoder, StandardScaler
Serialization	Joblib	≥ 1.2.0	Model artifact persistence
Agent Framework	LangGraph	StateGraph	Multi-node workflow orchestration
Vector Store	FAISS	In-memory	Similarity-based document retrieval
Embeddings	HuggingFace	all-MiniLM-L6-v2	Text-to-vector encoding
LLM Provider	Groq API	Cloud-hosted	Fast LLM inference
LLM Model	LLaMA 3.3	70B Versatile	Reasoning + structured generation

Layer	Technology	Version / Model	Purpose
Knowledge Base	JSON	9 curated entries	Domain-specific retention strategies
Environment	python-dotenv	—	Secure API key management

Project File Structure

```
project/
■■■ app.py # Main application (~750 lines)
■■■ retention_knowledge.json # RAG knowledge base (9 strategies)
■■■ model_test.py # Model validation script
■■■ requirements.txt # Python dependencies
■■■ .env # API keys (GROQ_API_KEY)
■■■ .streamlit/ # Streamlit config
■■■ notebook_&_otherpkl/
■ ■■■ final_churn_model.pkl # Trained XGBoost model
■ ■■■ scaler.pkl # StandardScaler (3 numeric columns)
■ ■■■ threshold.pkl # Optimized threshold (0.4)
■ ■■■ encoders.pkl # LabelEncoders (16 categorical cols)
■ ■■■ feature_order.pkl # Column ordering from training
■■■ Raw_Dataset/ # Original Telco churn CSV
■■■ EDA Insights/ # Exploratory data analysis outputs
■■■ Report/ # Project documentation
```

5. Agent Workflow Using LangGraph

Why LangGraph?

LangGraph was chosen over alternatives (vanilla LangChain Agents, raw function chaining) because the retention strategy pipeline has a **fixed, known structure**. There is no value in letting the LLM dynamically decide what to do next when the optimal path is always: assess risk → retrieve strategies → generate plan.

Factor	LangChain Agent	LangGraph (Chosen)
Execution flow	Dynamic — LLM decides next step	Deterministic — edges define flow
Predictability	Low — paths may vary per run	High — always Risk→Retrieval→Planning
Debuggability	Hard — internal reasoning is opaque	Easy — inspect state at each node
Latency	Higher — multiple routing LLM calls	Lower — only 1 LLM call (planning)
Suitability	Open-ended tasks	Well-defined, fixed pipelines

Graph Construction

```
from langgraph.graph import StateGraph

builder = StateGraph(AgentState)

# Register the three processing nodes
builder.add_node("risk", risk_node)
builder.add_node("retrieval", retrieval_node)
builder.add_node("planning", planning_node)

# Define the execution order (deterministic pipeline)
builder.set_entry_point("risk")
builder.add_edge("risk", "retrieval")
builder.add_edge("retrieval", "planning")

# Compile into an executable graph
graph = builder.compile()
```

Agent Workflow Diagram

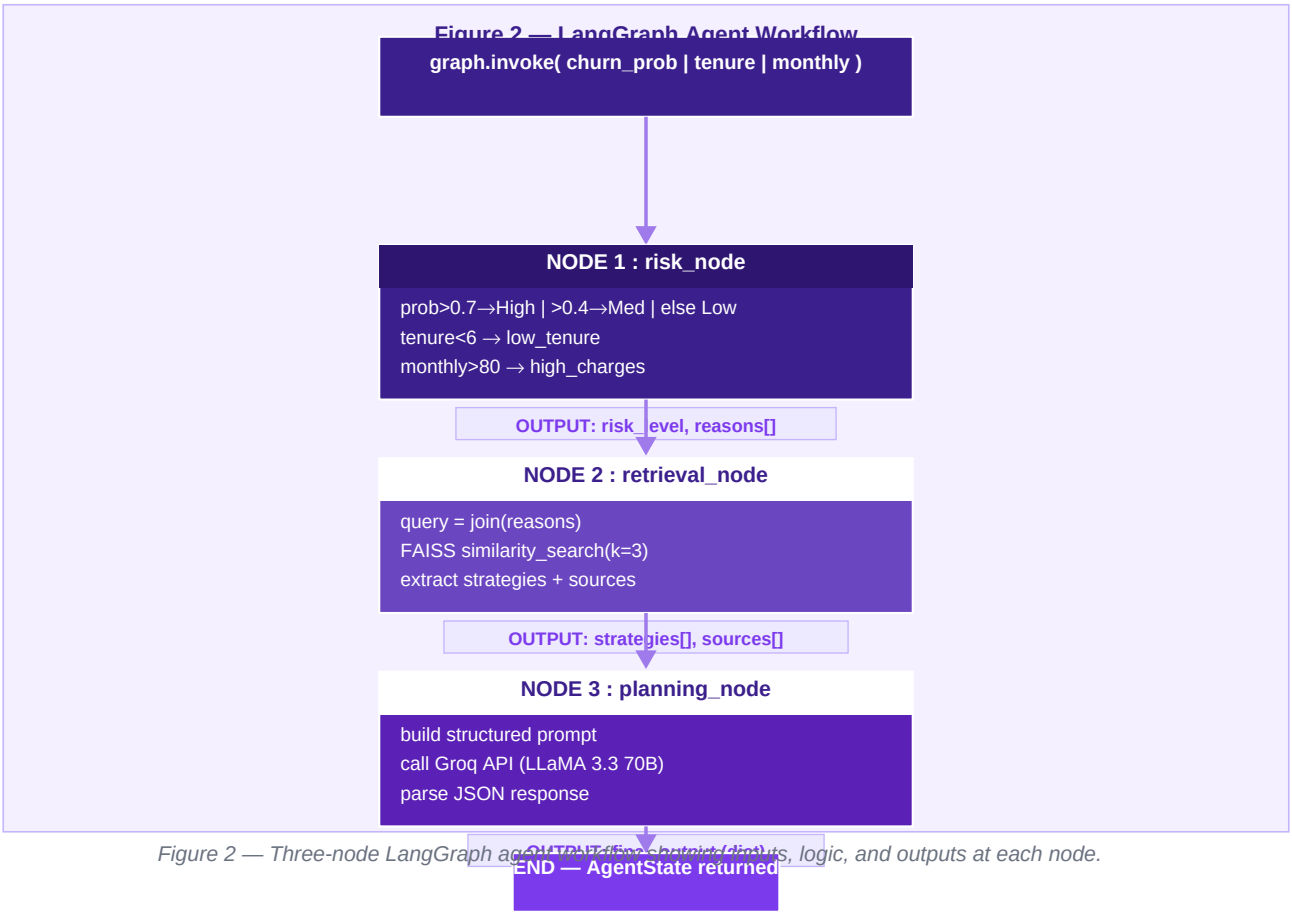

Figure 2 — Three-node LangGraph agent workflow showing inputs, logic, and outputs at each node.

6. Agent State Design

The **AgentState** is a Python TypedDict that serves as the shared data contract between all agent nodes. It is passed through the pipeline immutably — each node reads from it and returns a new dictionary with additional fields appended, never mutating the original state.

State Definition

```
from typing import TypedDict, List

class AgentState(TypedDict):
# --- Inputs (set before graph.invoke) ---
churn_prob : float # ML output: P(churn) in [0.0, 1.0]
tenure : int # Customer tenure in months (0-72)
monthly : float # Monthly charges in USD ($18-$120)

# --- Set by risk_node ---
risk_level : str # "High" | "Medium" | "Low"
reasons : List[str] # e.g. ["low_tenure", "high_charges"]

# --- Set by retrieval_node ---
strategies : List[str] # Retrieved retention strategies from FAISS
sources : List[str] # Academic/industry source citations

# --- Set by planning_node ---
final_output : str # Structured JSON from LLM (dict at runtime)
```

Field Reference

Field	Type	Set By	Description
churn_prob	float	Caller	Raw churn probability from XGBoost. Range: 0.0–1.0.
tenure	int	Caller	Customer tenure in months. Trigger threshold: < 6.
monthly	float	Caller	Monthly charges in USD. Trigger threshold: > $80.
risk_level	str	risk_node	>0.7=High | >0.4=Medium | ≤0.4=Low
reasons	List[str]	risk_node	Root cause tags: low_tenure, high_charges, general
strategies	List[str]	retrieval_node	Top-3 FAISS-retrieved retention strategy texts
sources	List[str]	retrieval_node	Corresponding citation strings for each strategy
final_output	dict	planning_node	LLM JSON: risk_summary, recommendations, sources, disclaimer

State Flow Through Pipeline

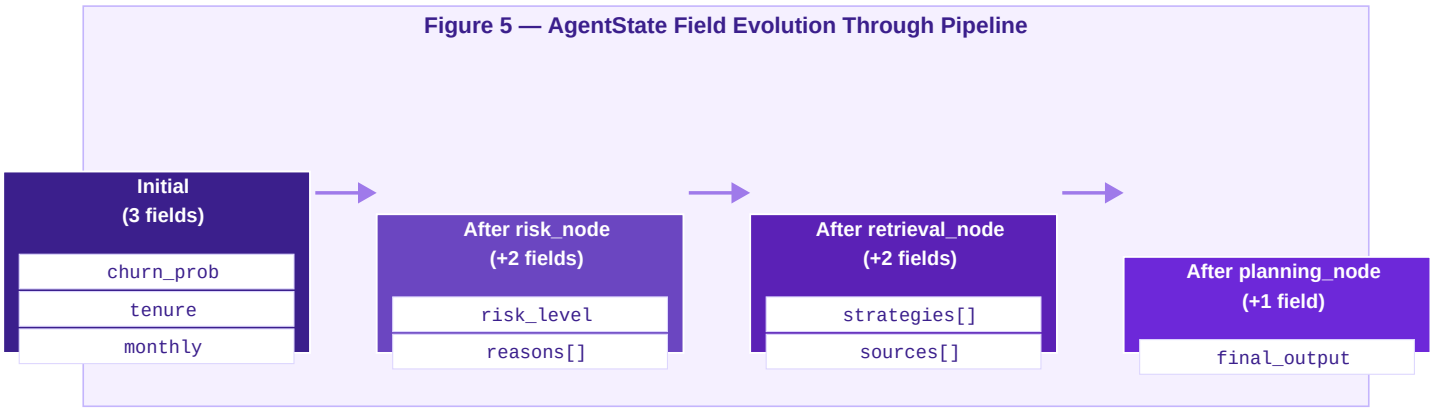

Figure 5 — AgentState evolves from 3 input fields to 8 fields as each node appends its outputs.

7. Detailed Node Explanations

7.1 Node 1 — Risk Assessment (risk_node)

Purpose: Convert the raw churn probability into a human-interpretable risk level and identify the specific reasons contributing to churn risk. A raw probability like `0.73` is not actionable for business stakeholders. This node transforms it into a categorical label ('High') and identifies root causes ('low_tenure', 'high_charges') that drive the RAG retrieval query.

Code Implementation

```
def risk_node(state: AgentState) -> AgentState:
prob = state['churn_prob']

# Classify risk level
if prob > 0.7:
risk = "High"
elif prob > 0.4:
risk = "Medium"
else:
risk = "Low"

# Identify churn reasons from customer attributes
reasons = []
if state['tenure'] < 6:
reasons.append("low_tenure")
if state['monthly'] > 80:
reasons.append("high_charges")
if not reasons:
reasons.append("general") # Catch-all fallback

return {**state, 'risk_level': risk, 'reasons': reasons}
```

Risk Classification Logic

Condition	Risk Level	Business Rationale
prob > 0.7	HIGH	Strong churn signal — immediate intervention required
0.4 < prob ≤ 0.7	MEDIUM	Moderate risk — proactive engagement recommended
prob ≤ 0.4	LOW	Stable customer — maintain standard support level

Attribute Check	Reason Tag	Industry Rationale
tenure < 6 months	low_tenure	Highest churn rates occur within first 6 months
monthly > $80	high_charges	High cost is the top driver of voluntary telecom churn

Attribute Check	Reason Tag	Industry Rationale
Neither condition	general	Catch-all for customers without specific risk triggers

■ **Why rule-based and not a secondary ML model?** Transparency (stakeholders see exactly why a risk was assigned), Speed (no additional API latency), and Alignment with RAG (reason tags directly correspond to condition fields in the knowledge base).

7.2 Node 2 — Strategy Retrieval (retrieval_node)

Purpose: Retrieve the most relevant retention strategies from a curated knowledge base using vector similarity search. This grounds the final LLM output in verified domain knowledge, preventing hallucination.

Without RAG, the LLM would generate strategies from its parametric training data — generic, unverifiable, and potentially fabricated. By constraining the LLM to retrieved strategies with known sources, the system produces **trustworthy, citation-backed recommendations**.

Code Implementation

```
def retrieval_node(state: AgentState) -> AgentState:
# Convert reason tags into a natural-language search query
query = " ".join(state["reasons"]) # e.g. "low_tenure high_charges"

# Perform cosine-similarity search against the FAISS vector store
results = vectorstore.similarity_search(query, k=3)

# Extract strategy text and source citations
strategies, sources = [], []
for doc in results:
strategies.append(doc.page_content)
sources.append(doc.metadata["source"])

return {
**state,
'strategies': list(set(strategies)), # deduplicate
'sources': list(set(sources)),
}
```

■ **Why k=3?** k=1 is too narrow (may miss complementary strategies). k=5 introduces noise on a 9-document knowledge base. k=3 balances coverage and precision. **list(set(...))** deduplication ensures the LLM prompt is not inflated with redundant content.

7.3 Node 3 — Planning & Recommendation (planning_node)

Purpose: Synthesise the risk assessment and retrieved strategies into a coherent, structured, human-readable retention plan. The LLM acts as a **reasoning layer** that interprets risk context, selects the most relevant strategies, and packages them into a structured JSON output.

Code Implementation

```
def planning_node(state: AgentState) -> AgentState:
prompt = f'''
You are an AI Customer Retention Strategist.
Customer churn probability: {state['churn_prob']}
Risk level: {state['risk_level']}
Reasons: {state['reasons']}
Retrieved Strategies: {state['strategies']}
Sources: {state['sources']}

IMPORTANT RULES:
- Use ONLY the provided strategies and sources
- Do NOT generate new strategies
- If no relevant strategy, say "No recommendation found"

STRICT OUTPUT FORMAT (JSON ONLY):
{
"risk_summary": "short explanation of churn risk",
"recommendations": ["action 1", "action 2", "action 3"],
"sources": ["source1", "source2"],
"disclaimer": "This prediction is probabilistic..."
}
ONLY return valid JSON. No extra text.
'''

response = client.chat.completions.create(
model="llama-3.3-70b-versatile",
messages=[{"role": "user", "content": prompt}]
)
raw = response.choices[0].message.content

try:
parsed = json.loads(raw)
except Exception:
parsed = {"risk_summary": "Parsing error",
"recommendations": [], "sources": [],
"disclaimer": "Model output could not be parsed"}

return {**state, 'final_output': parsed}
```

8. RAG Pipeline Explanation

Retrieval-Augmented Generation (RAG) enhances LLM outputs by first retrieving relevant information from an external knowledge base, then injecting that information as context into the LLM prompt. This grounds the LLM's response in verified data rather than relying solely on its parametric knowledge, directly preventing hallucination.

Knowledge Base Structure

The knowledge base (`retention_knowledge.json`) contains 9 curated retention strategies organised by churn condition:

Condition Tag	Count	Target Customer Profile
low_tenure	2	Early-lifecycle customers with tenure < 6 months
high_charges	2	Cost-sensitive customers with monthly charges > $80
low_engagement	2	Disengaged customers not using available services
no_support	1	Customers without access to tech support
general	2	Broadly applicable retention strategies

Sources include: Harvard Business Review, McKinsey, Bain & Company, HubSpot, HBS Working Knowledge, and MDPI.

Document Conversion

```
from langchain_core.documents import Document

docs = []
for item in knowledge:
content = f"Condition: {item['condition']}\nStrategy: {item['strategy']}"
docs.append(Document(
page_content=content, # Searchable text (embedded)
metadata={"source": item["source"],
"condition": item["condition"]} # Citation + tag
))
```

Embedding Model

```
from langchain_community.embeddings import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

Model	all-MiniLM-L6-v2
Provider	HuggingFace / Sentence-Transformers
Embedding Dimension	384
Training Objective	Semantic similarity
Inference Speed	~14,000 sentences / second on CPU
Why chosen	Lightweight, GPU-free, excellent short-text similarity

FAISS Vector Store & Similarity Search

```
from langchain_community.vectorstores import FAISS

# Build index at app startup
vectorstore = FAISS.from_documents(docs, embedding_model)

# Retrieve at inference time
query = " ".join(state["reasons"]) # "low_tenure high_charges"
results = vectorstore.similarity_search(query, k=3)
```

RAG Pipeline Diagram

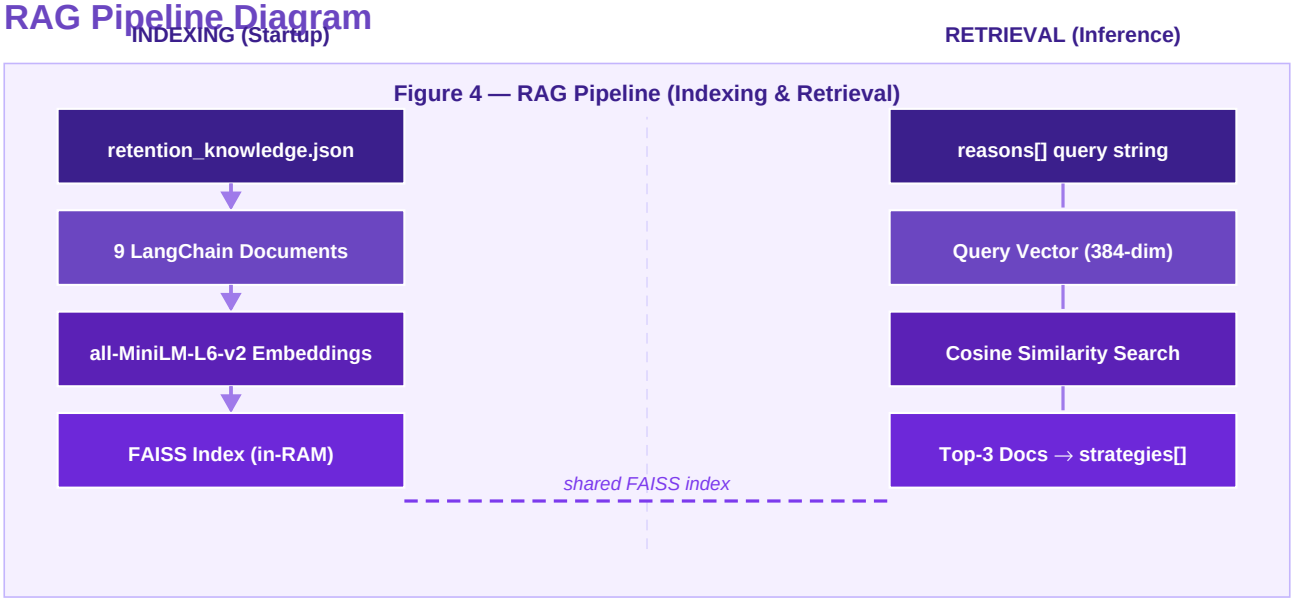

Figure 4 — RAG Pipeline showing FAISS indexing at startup and cosine-similarity retrieval at inference time.

9. Prompt Engineering Strategy

The prompt is carefully engineered to prevent hallucination, enforce structured output, and provide the LLM with complete context to reason accurately about each customer's situation. Three distinct strategies are layered together.

9.1 Anti-Hallucination Design

Rule	Constraint Text	Purpose
Source-bound	Use ONLY the provided strategies and sources	Forces LLM to use retrieved content only, not parametric knowledge
No fabrication	Do NOT generate new strategies	Explicitly prohibits inventing new recommendations
Graceful absence	If no relevant strategy, say 'No recommendation found'	Prevents hallucination when no strategy matches

9.2 Structured Context Injection

The prompt injects the full agent state as typed, labelled context, giving the LLM everything it needs without requiring inference or guesswork:

```
Customer churn probability: {state['churn_prob']} <- numeric precision
Risk level: {state['risk_level']} <- categorical context
Reasons: {state['reasons']} <- root cause tags
Retrieved Strategies: {state['strategies']} <- RAG grounding
Sources: {state['sources']} <- citation data
```

9.3 Output Format Enforcement

The use of **'STRICT'** and **'ONLY'** keywords signals high priority. The exact JSON template with key names and value types prevents the LLM from wrapping output in markdown fences or adding commentary. `'No extra text'` is an explicit instruction the 70B model reliably honours.

```
STRICT OUTPUT FORMAT (JSON ONLY):
{
"risk_summary": "1-2 sentence explanation of churn risk",
"recommendations": ["action 1", "action 2", "action 3"],
"sources": ["source citation 1", "source citation 2"],
"disclaimer": "probabilistic warning"
}
ONLY return valid JSON. No extra text.
```

10. LLM Output Structure

The LLM's response is parsed from JSON into a Python dictionary and rendered directly in the Streamlit UI. Each field serves a specific purpose in communicating the retention strategy to business users.

Field	Type	Purpose
risk_summary	string	1-2 sentence human-readable explanation of the customer's churn risk
recommendations	string[]	2-3 specific, actionable retention strategies from the knowledge base
sources	string[]	Academic/industry citations backing each recommendation
disclaimer	string	Probabilistic warning: predictions are not guarantees

UI Rendering Code

```
output = result["final_output"]

st.subheader("Risk Summary")
st.write(output["risk_summary"])

st.subheader("Recommendations")
for rec in output["recommendations"]:
st.write("•", rec)

st.subheader("Sources")
for src in output["sources"]:
st.write("-", src)

st.subheader("Disclaimer")
st.info(output["disclaimer"])
```

11. System Robustness & Error Handling

The system implements defensive programming at every layer, ensuring that partial failures (missing model files, invalid inputs, malformed LLM output) degrade gracefully rather than causing unhandled exceptions.

11.1 Model Loading Safety

```
@st.cache_resource
def load_model():
try:
return joblib.load("notebook_&_otherpkl/final_churn_model.pkl")
except Exception:
st.error("Error loading model file")
st.stop() # Halt gracefully — prevent silent failures
```

11.2 Input Validation

```
if monthly < 0 or total_c < 0:
st.warning("Charges cannot be negative.")
st.stop()
if tenure < 0:
st.warning("Tenure cannot be negative.")
st.stop()
```

11.3 LLM Output Parsing — Graceful Fallback

LLMs occasionally produce malformed JSON (trailing commas, markdown wrapping, extra prose). The fallback dict shares the same schema as a successful response, so the UI rendering code works without any branching logic:

```
try:
parsed_output = json.loads(raw_output)
except Exception:
parsed_output = {
"risk_summary": "Parsing error — please retry",
"recommendations": [],
"sources": [],
"disclaimer": "Model output could not be parsed"
}
```

11.4 Performance Caching

All heavy resources are cached with `@st.cache_resource`, preventing repeated loading on each Streamlit rerun. The FAISS index and embedding model are initialised once at module level, avoiding redundant computation.

11.5 API Key Security

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.getenv('GROQ_API_KEY')) # Never hardcoded
```

12. End-to-End Flow

The complete request lifecycle, from user interaction to displayed result, traverses 13 discrete steps across six system layers:

Step	Component	Action	Output
1	Streamlit UI	User fills 19 customer profile fields	Raw input values
2	Streamlit UI	User clicks 'Run Churn Prediction'	Triggers pipeline
3	Input Validation	Check for negative charges / tenure	Pass or halt with warning
4	LabelEncoders	Encode 16 categorical features	Numerical encoded values
5	Feature Ordering	Reorder columns to match training order	Aligned DataFrame
6	StandardScaler	Scale tenure, MonthlyCharges, TotalCharges	Standardised features
7	XGBoost Model	predict_proba(input_df)[0][1]	churn_prob in [0.0, 1.0]
8	graph.invoke()	Pass {churn_prob, tenure, monthly} to LangGraph	Initiates agent pipeline
9	risk_node	Classify risk level + identify churn reasons	risk_level, reasons[]
10	retrieval_node	FAISS similarity search with reasons as query (k=3)	strategies[], sources[]
11	planning_node	Build prompt → Groq API → parse JSON	final_output (dict)
12	Threshold Check	prob >= 0.4 → Churn / No Churn binary label	Classification result
13	Streamlit UI	Render progress bar, scores, recommendations, sources	Complete visual output

13. Example Input & Output

13.1 High-Risk Customer Profile

Feature	Value
Gender	Female
Senior Citizen	No
Tenure	4 months ← triggers low_tenure
Internet Service	Fiber optic
Contract	Month-to-month
Payment Method	Electronic check
Monthly Charges	$95.00 ← triggers high_charges
Total Charges	$380.00

13.2 ML Prediction Output

```
Churn Probability : 0.82
Threshold : 0.40
Prediction : Customer WILL churn
Confidence Score : 0.64
```

13.3 Agent Pipeline Execution

risk_node output

```
{
"risk_level": "High",
"reasons": ["low_tenure", "high_charges"]
}
```

retrieval_node output

```
{
"strategies": [
"Condition: low_tenure\nStrategy: Implement strong onboarding and early
engagement programs to help customers realise value quickly",
"Condition: low_tenure\nStrategy: Deliver value early in the lifecycle
to prevent early-stage churn",
"Condition: high_charges\nStrategy: Offer flexible pricing plans or
targeted discounts to reduce cost burden"
],
"sources": [
"Customer Onboarding Best Practices - HubSpot (Link 11)",
"The Value of Keeping the Right Customers - Harvard Business Review",
"Managing Churn to Maximize Profits - HBS Working Knowledge"
]
}
```


planning_node output (LLM-generated)


```
{
"risk_summary": "This customer has a high churn probability of 82%.
Primary risk factors: short tenure (4 months) and high monthly
charges ($95.00) - both strong indicators of imminent churn.",
"recommendations": [
"Implement a personalised onboarding program to ensure the customer
quickly realises the value of their current plan",
"Deliver early value through complimentary premium features during
the first 6 months to build loyalty",
"Offer a targeted pricing adjustment or loyalty discount to reduce
the perceived cost burden of the $95/month plan"
],
"sources": [
"Customer Onboarding Best Practices - HubSpot (Link 11)",
"The Value of Keeping the Right Customers - HBR (Link 1)"
],
"disclaimer": "This prediction is probabilistic and may not guarantee
actual churn."
}
```

14. Key Design Decisions

14.1 Why LangGraph over LangChain Agents?

The retention pipeline has a fixed, known structure. Letting an LLM dynamically route between nodes adds latency, unpredictability, and debugging difficulty with no benefit. LangGraph's explicit edges enforce the deterministic order: Risk → Retrieval → Planning.

14.2 Why Groq (LLaMA 3.3 70B) over Google Gemini?

Factor	Gemini	Groq + LLaMA 3.3 70B (Chosen)
Latency	Moderate	Very fast (Groq custom GroqChip hardware)
Cost	Pay-per-token	Generous free tier available
JSON compliance	Good	Strong — 70B model reliably follows structured output
Setup	Requires Google Cloud	Simple API key via python-dotenv

14.3 Why FAISS over ChromaDB / Pinecone?

Factor	Pinecone	ChromaDB	FAISS (Chosen)
Hosting	Cloud	Local	Local (in-memory)
Cost	Paid	Free	Free
Setup	API + acct	pip install	pip install
Performance	High	Good	High (microsecond for 9 docs)
Persistence	Built-in	Built-in	Manual (rebuilt at startup)

14.4 Why Classification Threshold = 0.4 instead of 0.5?

The default 0.5 threshold balances precision and recall equally. This system uses **0.4** to prioritise **recall** — it is more costly in telecom to miss a churning customer (false negative) than to flag a stable customer (false positive). A lower threshold enables proactive intervention earlier in the customer lifecycle.

15. Limitations

■ Knowledge Base Scale

The current knowledge base contains only 9 entries across 5 condition categories. This limits recommendation diversity for edge cases or multi-factor churn scenarios.

■ Reason Detection Scope

Only two customer attributes (tenure, monthly charges) are checked for reason tagging. Contract type, internet service, and tech support access are not captured as distinct reason tags.

■ Single LLM Call, No Validation

The planning node makes a single call without a self-critique or refinement loop. Suboptimal outputs (e.g., repeated recommendations) are not caught.

■ No Feedback Loop

The system does not track whether recommended strategies were acted upon or were effective. There is no mechanism for online learning from retention outcomes.

■ In-Memory FAISS

The FAISS index is rebuilt from scratch on every Streamlit app restart. For larger knowledge bases this would become a startup bottleneck.

■ LLM Availability Dependency

The planning node requires Groq API availability and internet connectivity. Free-tier rate limits may affect high-volume production usage.

16. Future Enhancements

■ Expanded Knowledge Base

Grow from 9 to 50+ strategies covering additional conditions: contract type, payment method, customer demographics. Support multi-source ingestion (PDFs, research papers, CRM data).

■ SHAP-Based Reason Detection

Replace hand-coded attribute rules with SHAP (SHapley Additive Explanations) values from the XGBoost model to dynamically identify the top contributing features per prediction and map them to condition tags.

■ Multi-Agent Architecture

Add a Validation Node (checks recommendation quality and source alignment) and a Personalization Node (segments customers and tailors strategies by cohort).

■ Persistent FAISS Index

Save and load the FAISS index from disk to eliminate cold-start rebuild overhead. Implement incremental indexing for growing knowledge bases.

■ Feedback Integration + RLHF

Track which recommendations were acted upon and their retention outcomes. Use Reinforcement Learning from Human Feedback (RLHF) to improve strategy selection over time.

■ Multi-Model Support & Batch Processing

Support switching between LLM providers (Groq, Gemini, OpenAI) with automatic fallback chains. Enable bulk CSV upload with parallel agent execution and aggregate business intelligence reporting.

17. Conclusion

The **Customer Churn Intelligence System** demonstrates the successful integration of classical machine learning with modern agentic AI architecture. By combining XGBoost for accurate churn probability estimation, LangGraph for deterministic stateful agent orchestration, FAISS and HuggingFace Embeddings for grounded knowledge retrieval, and Groq (LLaMA 3.3 70B) for structured reasoning and recommendation generation, the system transforms a simple binary prediction into actionable, source-backed business intelligence.

The three-node agent pipeline (Risk → Retrieval → Planning) ensures that every recommendation delivered to a business stakeholder is:

- **Contextually relevant** — informed by the specific customer's risk profile, not a generic template.
- **Evidence-grounded** — sourced from a curated domain knowledge base, not hallucinated by the LLM.

- **Actionable** — formatted as clear, implementable retention strategies that CRM teams can execute immediately.
- **Transparent** — every recommendation includes its source citation, supporting audit and trust.

This architecture serves as a reusable blueprint for building production-grade AI systems across any domain where **trustworthiness, explainability, and practical utility** are non-negotiable requirements — from financial risk advisory to healthcare decision support to e-commerce personalisation.

Document Version: 1.0 **Date:** April 2026
Stack: Python · Streamlit · XGBoost · LangGraph · FAISS · Groq · LLaMA 3.3 70B

18. References

The following sources form the knowledge base for the RAG pipeline and inform the design decisions documented in this report:

[1] Reichheld, F.F. (2001). The Value of Keeping the Right Customers. *Harvard Business Review.*

[2] Gupta, S. et al. Managing Churn to Maximize Profits. *HBS Working Knowledge.*

[3] Bain & Company. Customer Retention Economics and Growth Strategies.

[4] McKinsey & Company. Reducing Customer Churn in Telecommunications.

[5] MDPI. Machine Learning Approaches to Telecom Customer Churn Prediction. *Applied Sciences.*

[6] HubSpot. Customer Onboarding Best Practices.

[7] LangGraph Documentation. StateGraph and Agent Orchestration. LangChain Inc., 2024.

[8] Johnson, J. et al. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data.* (FAISS)

[9] Reimers, N. & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019.* (all-MiniLM-L6-v2)

[10] Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD 2016.*

[11] Groq Inc. (2024). Groq API Documentation — LLaMA 3.3 70B Versatile Model.

[12] IBM Telco Customer Churn Dataset. IBM Sample Data Sets.